

100 Days of Machine Learning - Day 1:

What is Machine Learning?

1. Introduction to the Series

The teacher starts by announcing a new series called "100 Days of Machine Learning." This course is designed to take a student from a beginner level to an advanced/intermediate level.

Why this course?

- Most people learn Machine Learning algorithms but don't know how to build a project from start to finish.
- This series focuses on the **Machine Learning Life Cycle** (End-to-End process).
- It covers everything: EDA (Exploratory Data Analysis), Feature Engineering, Model Selection, Deployment, and MLOps.

2. What is Machine Learning? (Definition)

Formal Definition:

"Machine Learning is a field of computer science that gives computer systems the ability to learn with data without being explicitly programmed."

Detailed Breakdown:

1. **Field of CS:** It is a branch of Computer Science.
2. **Ability to Learn:** Instead of humans telling the computer what to do, the computer learns by itself.
3. **Without Explicit Programming:** You don't write hundreds of "if-else" conditions for every scenario.

3. Conventional Programming vs. Machine Learning

The teacher explains the core difference in how logic is generated.

A. Conventional (Traditional) Programming

In normal software development, a human programmer creates the logic.

- **Input:** You give Data and a Program (Logic).
- **Process:** The computer runs the data through that specific logic.
- **Output:** You get the result.

Logic Flow: Data + Program (Logic) —> Computer —> Output

B. Machine Learning

In Machine Learning, the computer creates the logic based on the data provided.

- **Input:** You give Data (both the input and the expected output/answers).
- **Process:** The Machine Learning algorithm studies the data and finds "patterns."
- **Output:** The computer gives you the **Program (Logic)** itself.

Logic Flow:

Data + Output —> Computer —> Program (Logic)

4. Practical Example: Adding Numbers

To make the concept simple, the teacher uses an addition example.

Traditional Approach:

You write a code to add two numbers.

Full Code

```
def add_two_numbers(a, b):  
    return a + b
```

Line-by-line Explanation:

- `def add_two_numbers(a, b):` -> We define a function that expects exactly two inputs, a and b.
- `return a + b` -> We explicitly tell the computer to perform the "+" operation.
- **Limitation:** If you give this program 3 numbers or 10 numbers, it will fail because it is "explicitly programmed" only for two.

Machine Learning Approach:

You don't tell the computer to "add." Instead, you give it a file with thousands of examples:

- Input: 2, 2 | Output: 4
- Input: 5, 3 | Output: 8
- Input: 10, 20 | Output: 30

How it works:

- The ML model looks at these thousands of examples.
- It realizes the pattern: *"Whenever these numbers come in, the output is their sum."*
- It learns the "Addition Logic" by itself.
- **Advantage:** In the future, even if the data format changes slightly, the model can adapt because it understands the underlying pattern, not just a fixed line of code.

5. When is Machine Learning Useful?

The teacher identifies four main scenarios where ML is better than traditional software.

Scenario 1: Logic is too complex to write (e.g., Spam Filter)

If you try to write a spam filter using if-else (e.g., if "discount" exists, then spam), spammers will just change the word "discount" to "d-iscount." You would have to constantly update your code.

- **ML Solution:** The model learns from new spam emails automatically and updates its own logic.

Scenario 2: Problems humans learn by experience (e.g., Image Recognition)

How do you explain to a computer what a "Dog" looks like? Dogs have different sizes, colors, and breeds. Writing if-else for every breed is impossible.

- **ML Solution:** Like a child, the computer looks at thousands of pictures of dogs and learns the features (ears, nose, tail) that define a dog.

Scenario 3: Changing Environments

If the nature of the data changes over time (like stock markets or user preferences), a fixed program will fail. ML models can be retrained on new data to stay relevant.

Scenario 4: Data Mining (Finding Hidden Patterns)

Sometimes data contains patterns that humans can't see even by looking at graphs.

- **Example:** A bank might find a hidden pattern that people who buy a certain type of bread are more likely to default on loans. ML helps discover these "non-obvious" insights.

6. The History of Machine Learning (The "Nawazuddin Siddiqui" Analogy)

The teacher uses a famous Bollywood actor to explain ML's history.

- **The Point:** Machine Learning is NOT new. It has existed for 40-50 years.
- **The Analogy:** Nawazuddin Siddiqui was in the industry for a long time (even in small roles like in *Munna Bhai MBBS*), but he became a "superstar" only after 2010.
- **Why ML became a superstar after 2010?**
 1. **Data Availability:** With smartphones and the internet, we now generate massive amounts of data.
 2. **Hardware Power:** We now have powerful GPUs and high RAM that can process complex algorithms which wasn't possible in the 1980s.

7. The Job Market and Future Trends

- **Current State:** High Demand + Low Supply of Talent = Very high salaries for ML Engineers.
- **The Future (5-10 years):** Eventually, ML will be taught in every college. Everyone will know it (like Java/C++ today). When supply increases, the "crazy" high salaries will normalize.
- **Advice:** We are currently in the "Growth Phase." If you learn it well now, you can build a very successful career before the market becomes saturated.

Key Points to Remember

- **ML = Data + Patterns.**
- You don't write the logic; the machine finds the logic.
- ML is most powerful when you have **Big Data** and **Complex Problems** that if-else cannot solve.
- Focus on the **Lifecycle** (End-to-end), not just the math of the algorithms.